DOMAIN 02

Web Development

Technical Club Recruitment Task: Website Development | Choose tasks by your year

FIRST-YEAR STUDENTS

Objective: Build a creative and visually appealing website using HTML and CSS, with optional JavaScript. This evaluates your creativity, web development fundamentals and problem-solving skills. Choose **ONE** task.

Task	Requirements
1. Personal Portfolio Website	Create a personal website to introduce yourself and showcase your interests. Sections: Home with your name and introduction, About Me, Skills and interests, Projects or achievements (if any), Contact or social links.
2. Mini Games Website	A simple website with browser-based games: attractive homepage, at least one playable game, clear instructions and result display, a restart button. Suggested: Rock Paper Scissors, Tic-Tac-Toe, Number Guessing Game.
3. Gamified Portfolio Website	Turn a personal portfolio into a game-inspired experience: personal introduction and profile, skills as levels or achievements, a progress bar or achievement badges, at least one interactive feature.
4. Animated Website	Show creativity with CSS animations: attractive hero section, at least two CSS animations, hover effects on buttons or cards, creative typography, colors and layout.

TECHNICAL REQUIREMENTS

- Use HTML5 and CSS3.
- JavaScript is optional, except where required for game functionality or interactive features.
- No frameworks or libraries such as React, Bootstrap or Tailwind CSS.
- Focus on clean code, good design and working features.

LIVE DEMONSTRATION GUIDELINES

- Bring your laptop with your project ready to run.
- Demonstrate your project live and explain its key features.
- Be prepared to explain your code, technology choices and implementation.
- If deployed, you may also showcase the live website.
- Keep your GitHub repository and README ready for reference.

> **SUBMISSION: BRING EVERYTHING ON THE DAY OF AUDITION.** There is **no online submission**: do not send or upload your project anywhere. Bring your laptop with the project ready to run, and keep your GitHub repository, README and deployed link (if any) ready to show.

IMPORTANT NOTES

- Choose only ONE task.
- You are not expected to know everything beforehand. Focus on learning and implementing the basics.
- Use of AI is allowed, but you must be able to debug and change small functionalities (if asked) during the interview.
- A simple, well-executed project is better than an incomplete, complicated one.
- Deployment is encouraged but not compulsory.

All the best! Be creative, experiment, and have fun building your first website.

SECOND-YEAR STUDENTS

Choose any **ONE** of the three tasks.

Task 1: Operation Hidden Code (Frontend Development)

Tech stack: HTML, CSS, JavaScript

Build a futuristic mystery website where users explore an interactive interface, solve puzzles and uncover a secret code.

Mandatory:

- Creative UI with animations and visual effects.
- Interactive terminal with at least five custom commands.
- At least three connected puzzles.
- One hidden Easter egg.
- Final secret-code reveal.
- Feedback for correct and incorrect answers.

Bonus: Timer, scoring system, sound effects, saved progress, or multiple endings.
Bring on audition day: GitHub repository, deployed website and a short demo (all shown live on your laptop).

Task 2: Full-Stack Development (Build Your Own Product)

Tech stack: Open choice (MERN, Next.js, Django or any suitable stack)

Build a complete, functional web application that solves a real-world problem or provides an original user experience.

Mandatory:

- Interactive frontend.
- Backend APIs with meaningful business logic.
- Database integration.
- User authentication and appropriate access control.
- Input validation and error handling.
- At least one feature beyond basic CRUD operations.
- Deployed working application.

Suggested ideas:

- Collaborative workspace with real-time updates.
- Gamified coding platform with leaderboards.
- Campus community platform.
- Hackathon teammate-matching platform.
- An original application idea.

Bonus: AI integration, real-time communication, automated tests, or an admin dashboard.
Bring on audition day: GitHub repository, deployed application, README, and a demo explaining the architecture and technical decisions (all shown live on your laptop).

Task 3: Innovation Project (Think Beyond the Ordinary)

Tech stack: Open choice

Identify a real-world problem and develop an innovative solution using technology. It may involve web development, mobile applications, AI/ML, IoT, automation, cybersecurity or any other relevant field.

Requirements:

- Identify a genuine problem and its target users.
- Propose an original or meaningfully improved solution.

- Develop a working prototype or minimum viable product (MVP).
- Demonstrate the core functionality.
- Explain the technology used and why it was chosen.
- Describe the potential impact, limitations and future scope.

Bonus: A unique technical approach, user testing, measurable results, or a clear plan for real-world implementation.

LIVE DEMONSTRATION GUIDELINES

- Bring your laptop with your project ready to run.
- Demonstrate your project live and explain its key features.
- Be prepared to explain your code, technology choices and implementation.
- If deployed, you may also showcase the live website.
- Keep your GitHub repository and README ready for reference.

> **SUBMISSION: BRING EVERYTHING ON THE DAY OF AUDITION.** There is **no online submission**: do not send or upload your project anywhere. Bring your laptop with the project ready to run, and keep your GitHub repository, README and deployed link ready to show.

GENERAL GUIDELINES

- Choose any ONE of the three tasks.
- Task 1 must use HTML, CSS and JavaScript without frontend frameworks.
- Task 2 and Task 3 allow suitable technologies of your choice.
- Use of AI is allowed, but you must be able to debug and change small functionalities (if asked) during the interview.
- A working, well-explained project is valued more than unnecessary complexity.

Don't just build a project. Build something worth remembering.

THIRD-YEAR STUDENTS: ADVANCED DEVELOPMENT & INNOVATION

Choose **ONE** of the following challenges. Tech stack for all tasks: open choice.

Task 1: Build a Production-Ready Web Application

Design and develop a full-stack application that solves a real-world problem and demonstrates production-level engineering practices.

Mandatory requirements:

- Responsive and user-friendly interface.
- Frontend, backend and database integration.
- Authentication and role-based access control where appropriate.
- Proper API design, validation and error handling.
- Secure handling of user data and sensitive configuration.
- At least one advanced feature, such as real-time updates, background processing, analytics or intelligent recommendations.
- Deployment with a working demonstration.

Bonus: Automated tests, CI/CD, caching, logging, monitoring or containerization.

Task 2: AI-Powered Innovation Challenge

Build an AI-powered application that solves a meaningful problem using AI, machine learning, intelligent automation or data-driven decision-making.

Mandatory requirements:

- Clearly defined problem statement and target users.
- A working prototype with a meaningful AI/ML component.
- Integration with an application interface or workflow.
- Evaluation of output quality using suitable test cases or metrics.
- Handling of invalid inputs, failure cases and limitations.
- Explanation of the model, approach and technology choices.

Bonus: Retrieval-Augmented Generation (RAG), agentic workflows, local models, model evaluation, or cost and latency optimization.

Task 3: Open Innovation (Build Something Impactful)

Identify a real-world problem and create an original technological solution. You may work in domains such as healthcare, education, agriculture, cybersecurity, accessibility, sustainability, smart cities or developer tools.

Mandatory requirements:

- Identify the problem and intended users.
- Propose a solution with a clear innovation factor.
- Build a working prototype or minimum viable product (MVP).
- Demonstrate the core functionality using a realistic scenario.
- Explain the architecture, technology choices and implementation.
- Discuss feasibility, limitations and future improvements.

Bonus: User validation, measurable impact, novel algorithms, hardware integration, or a credible path to real-world adoption.

LIVE DEMONSTRATION GUIDELINES

- Bring your laptop with the project ready to run.

- Demonstrate the working application or prototype live.
- Explain the architecture, key implementation decisions and technical challenges.
- Be prepared to answer questions about your code and show how your solution handles realistic scenarios.
- Keep your GitHub repository, README and deployment link ready for reference, if applicable.

> **SUBMISSION: BRING EVERYTHING ON THE DAY OF AUDITION.** There is **no online submission**: do not send or upload your project anywhere. Bring your laptop with the project ready to run, and keep your GitHub repository, README, and deployment link (if applicable) ready to show.

IMPORTANT NOTES

- Choose only ONE task.
- You may use any suitable programming language, framework or technology.
- Use of AI is allowed, but you must be able to debug and change small functionalities (if asked) during the interview.
- A working, well-tested solution is more valuable than a feature-heavy but unreliable prototype.

The challenge: Don't just demonstrate what you know. Demonstrate what you can engineer.